

TECHNICAL PDF EDITION

Workflow Catalog

Operating, debugging, and extending The Ontology Machine workflows.

Core rule: workflows cross module boundaries; Kernel state controls the route, owner modules mutate state, and durable artifacts prove completion.

Prepared from 05_Workflow_Catalog.md. Revised for readability, clean tables, and a rendered workflow reference style.

Contents

1. First read: workflow mental model	3
2. Workflow Families.....	4
3. Common Workflow Invariants	5
4. Public Taxonomy Agent Tool Surface	6
5. Taxonomy Agent Launch Workflow.....	8
6. Direct Orchestrator Ingestion Workflow	10
7. Manual Pipeline Run Workflow.....	12
8. Ingestion Stage Micro-Workflows.....	14
9. Database Creation Workflows	15
10. Custom Taxonomy Authoring Workflow	18
11. Custom Projection Authoring Workflow	20
12. Semantic Release Publication And Activation Workflow	22
13. Embedding Generation Workflow	24
14. Database Rebuild From Artifacts Workflow.....	25
15. Database Reset Workflow.....	27
16. Additive Database Merge Workflow.....	28
17. Base Graph Mining Workflow.....	30
18. Ontology Agent Workflow	32
19. Query Agent Workflow	34
20. Error Case Inspection Workflow.....	36
21. Recovery And Resume Workflow	37
22. Testing And Verification Map	38

Recommended first pass: read sections 1-5, then jump to the workflow that matches the operation or failure mode.

1. First read: workflow mental model

This chapter describes the workflows a maintainer or support engineer must understand when operating, debugging or extending The Ontology Machine.

A workflow is not just a button in the UI. In this system a workflow is a bounded movement across module boundaries: user intent enters through the Frontend or Orchestrator, Kernel state records what is allowed to happen, owner modules perform the actual mutation, and the result is written back as artifacts, database rows, manifests, event streams or final notices.

The workflow catalog therefore uses the following separation:

- **Surface** means how the user or agent starts the work.
- **Kernel route** means the controlled workflow path and recovery state.
- **Owner** means the module that is allowed to mutate its own files or DB.
- **Artifacts** means durable evidence outside chat.
- **Expected end state** means the state that must be true before the workflow can be considered complete.

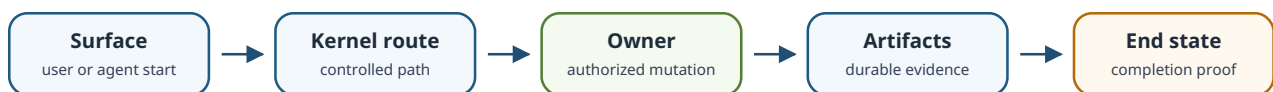


Figure 1. A workflow is a target-bound movement from user intent to durable completion evidence.

If prose, UI labels and code disagree, trust the executed dispatcher, validators and owner adapters first. Some older labels in manifests and final notice guidance still use names from earlier Kernel phases.

2. Workflow Families

Family	User-Facing Surface	Primary Owner	Durable Result
Direct ingestion	Orchestrator GUI	00 - Orchestrator plus downstream modules	Artifact tree content and Corpus DB rows
Agent-guided ingestion	Client Frontend Taxonomy Agent	08 - Semantic Control Kernel orchestrating Orchestrator and Corpus Builder	Batch manifests, progress events, final notice, Corpus DB rows
Database creation	Taxonomy Agent workflow tools	Kernel route plus Corpus Builder and Normalizer owners	Artifact tree, empty Corpus DB, Semantic Release state
Taxonomy and projection authoring	Taxonomy Agent continuation flow	Kernel route plus Orchestrator LLM authoring and Normalizer materialization	Custom taxonomy/projection components and Semantic Release package
Semantic Release activation	Creation, merge, rebuild and direct Orchestrator flows	Corpus Builder	Active release snapshot inside the Corpus DB
Database rebuild	Taxonomy Agent workflow tool	Kernel route plus Corpus Builder standalone rebuild	Rebuilt Corpus DB from artifact-level evidence
Additive database merge	Taxonomy Agent workflow tool	Kernel route plus Corpus Builder merge services	Target artifact tree, merged DB, merged Semantic Release
Database reset	Taxonomy Agent workflow tool	Kernel route plus Corpus Builder admin service	Empty materialized DB with release preserved
Base Graph mining	Ontology Agent tool and Kernel ontology primitive	Corpus Builder ontology workflow	Source-document and structural base relations
Ontology mining	Ontology Agent SQL tools	Client Frontend Ontology Agent and Corpus DB ontology layer	Versioned ontology lenses, nodes, edges, assertions and evidence
Querying	Query Agent	Client Frontend read tools	Answer text plus source reconciliation
Recovery	Taxonomy Agent support tools and Kernel dialogs	Kernel support control	Resumed, cancelled, blocked or completed workflow state

3. Common Workflow Invariants

Every workflow in this chapter obeys a few shared rules.

Owner mutation

The Kernel and Frontend do not silently rewrite owner state. The Corpus Builder owns Corpus DB creation, reset, rebuild, Base Graph mining and DB materialized rows. The Normalizer owns Semantic Release component materialization. The Orchestrator owns pipeline execution, raw stage artifacts and stage progress.

Target identity

Dangerous workflows must prove that the target artifact root and target database are still the same target selected by the user. This is why manual pipeline runs, rebuilds, resets and merges keep asking for confirmations or target-bound receipts instead of trusting loose paths from chat.

Artifact tree binding

The Corpus DB is expected to live under the selected artifact tree's `Corpus` area. Workflows that accept a DB path validate that the path stays inside the configured storage boundary. A DB outside that boundary is treated as a target identity error.

Semantic Release requirement

Ingestion needs an active Semantic Release. The Corpus Builder materializes normalized documents against the active taxonomy and projections. Empty DB creation can intentionally stop before release activation, but ingest cannot.

Progress is evidence, not completion

Stage progress and "process running" status are not final success. A pipeline run is complete only after the owner run has finalized, Kernel correlation has passed, final manifests have been written and a final notice has been emitted.

LLM boundaries

LLM calls are allowed in interpretation, normalization, taxonomy authoring, projection authoring and agent chat. Kernel route execution, target validation, DB reset, DB rebuild, DB merge mechanics and Base Graph mining are deterministic or owner-validated control work.

Recovery before improvisation

If a workflow blocks, the next step is `kernel_status`, `kernel_resume_state`, `kernel_continue_resumable_workflow` or `kernel_cancel_active_run`, not a new ad hoc path guessed from chat.

4. Public Taxonomy Agent Tool Surface

The Taxonomy Agent sees a permanent set of sixteen Kernel-facing tools. The Frontend dropdown intentionally exposes only the twelve workflow-start labels; the four support controls remain available to the agent but are not normal workflow menu entries.

Tool	Type	Purpose
<code>empty_database_no_semantic_release</code>	Start workflow	Create an artifact tree and empty Corpus DB without attaching a Semantic Release.
<code>empty_database_default_taxonomy_no_projections</code>	Start workflow	Create an empty DB with default taxonomy only; leave projections incomplete.
<code>empty_database_default_taxonomy_default_projections</code>	Start workflow	Create an empty DB with default taxonomy and default projections active.
<code>empty_database_default_taxonomy_custom_projections</code>	Start workflow	Create an empty DB with default taxonomy and user-authored projections.
<code>empty_database_custom_taxonomy_no_projections</code>	Start workflow	Create an empty DB with user-authored taxonomy only; leave projections incomplete.
<code>empty_database_custom_taxonomy_custom_projections</code>	Start workflow	Create an empty DB with user-authored taxonomy and user-authored projections active.
<code>manual_pipeline_run</code>	Start workflow	Ingest files into an existing artifact tree and active Corpus DB.
<code>database_merge_additive_only</code>	Start workflow	Merge two or more source databases into a target database without deleting source state.
<code>database_rebuild_from_artifacts</code>	Start workflow	Rebuild a Corpus DB from artifact tree evidence.
<code>create_custom_taxonomy_path</code>	Continuation workflow	Continue taxonomy authoring from a resumable creation state.
<code>create_custom_projection_path</code>	Continuation workflow	Continue projection authoring from a resumable creation state.

Tool	Type	Purpose
<code>reset_database</code>	Start workflow	Clear materialized Corpus content while preserving the active Semantic Release.
<code>kernel_status</code>	Support control	Inspect active runs, pending dialogs and resumable workflow state.
<code>kernel_resume_state</code>	Support control	List opaque resume options.
<code>kernel_continue_resumable_workflow</code>	Support control	Continue a resumable workflow by <code>resume_option_ref</code> .
<code>kernel_cancel_active_run</code>	Support control	Cancel Kernel-owned active workflow state.

The public tool inventory is anchored in:

- `08 - Semantic Control Kernel/semantic_control_kernel/surface/agent_tools.py`
- `08 - Semantic Control Kernel/semantic_control_kernel/services/agent_workflow_dispatcher.py`
- `Client Frontend/client_frontend/pipeline_agent/kernel_tool_surface.js`
- `Client Frontend/client_frontend/browser/main_app/taxonomy_workflow_launcher.ts`

5. Taxonomy Agent Launch Workflow

Purpose

Let the user start Kernel workflows through a conversational agent or through the workflow selector, while keeping the actual Kernel toolcall surface small and controlled.

Trigger

The user opens the Client Frontend, selects the Taxonomy Agent tab and either asks for a workflow in chat or selects a workflow from the dropdown.

User-facing surface

The dropdown does not call Kernel directly. It writes a normal chat command such as "Run Taxonomy Agent workflow `manual_pipeline_run`". The Taxonomy Agent then decides which visible MCP tool to call.

Preconditions

- Frontend configuration points to the Ontology Machine root.
- The Semantic Control Kernel MCP server can be discovered below the configured pipeline root.
- The local MCP bridge can spawn the Kernel server with a host bridge token.
- Any LLM-backed workflow has usable model credentials.

Steps

1. The browser creates a workflow command from the selected menu item or sends the user's typed Taxonomy Agent message.
2. The Frontend sends the chat request to the Taxonomy Agent route.
3. The Taxonomy Agent sees the permanent Kernel tool surface and selects a tool.
4. Client-side tool validation rejects model-authored arguments for empty-schema tools. Only `kernel_continue_resumable_workflow` accepts the opaque `resume_option_ref`.
5. The local MCP bridge forwards the call to the Kernel.
6. The Kernel dispatcher selects the route or support handler.
7. User interactions are emitted as Kernel dialogs and answered through host bridge routes, not through ordinary chat text.
8. Progress events are polled by the Frontend and displayed in the Kernel progress surface.
9. The Kernel emits a final notice or blocked state.

Produced artifacts and state

- Kernel event history and pending interaction state.

- Workflow-specific manifests under the target artifact tree.
- Final notice text mirrored back into the Taxonomy Agent conversation.

Human review points

The user must answer Kernel dialogs for target paths, destructive confirmation, sample selection, merge source selection and resume choices. Chat text alone is not a valid substitute for Kernel-owned confirmations.

Failure branches

- MCP server discovery fails.
- Host bridge token validation fails.
- The agent selects a continuation tool without a valid resume option.
- A Kernel dialog is stale or has already been resolved.
- The owner module returns `owner_error`.

Recovery strategy

Use `kernel_status` first. If a resumable option exists, use `kernel_resume_state` and then `kernel_continue_resumable_workflow`. If a run is stuck in an active Kernel state and no safe resume exists, use `kernel_cancel_active_run`.

Expected end state

The Taxonomy Agent answer should explain the final workflow state and any blocker. The Kernel event panel should no longer show a pending interaction unless the workflow intentionally stopped for user input.

6. Direct Orchestrator Ingestion Workflow

Purpose

Run the ingestion pipeline directly from the Orchestrator GUI without going through the Taxonomy Agent or Kernel workflow surface.

Trigger

The user creates or selects an artifact tree, chooses a Corpus DB, places input files under the input area and starts the Orchestrator run.

Preconditions

- Input files exist.
- The artifact tree is valid.
- The selected Corpus DB path is inside the configured Corpus storage area.
- A Semantic Release is active for the target DB or an explicit override path is used to activate one before ingest.
- Required model credentials are configured for the stages that need them.

Steps

1. Orchestrator UI validation checks input, artifact root, Corpus output folder and selected DB path.
2. Runtime Semantics loads the active release assets from the target DB.
3. The Optimizer converts each source into raw, locatable page evidence.
4. Request Enrichment prepares model requests for the Interpreter.
5. The Interpreter classifies and describes what is on each page or unit.
6. Projection hint validation checks whether Interpreter hints are supported by local structured signals.
7. The Validator checks shape, consistency and materialization safety.
8. The Normalizer maps interpreted content into the active taxonomy and projection shape.
9. The Corpus Builder materializes documents, pages, chunks, extracted fields, evidence atoms, page images and release-bound metadata into SQLite.
10. Embeddings run if configured and applicable.
11. Error cases and review flags are written as durable artifacts/state instead of disappearing into logs.

Produced artifacts

- Raw Optimizer outputs.

- Persisted OCR/model requests for applicable stages.
- Interpreter, Validator and Normalizer outputs.
- Error case bundles when a page or record cannot safely continue.
- Corpus DB rows and page-image evidence.
- Stage logs and progress snapshots.

Human review points

The direct Orchestrator path is operational rather than conversational. The user mostly intervenes before the run starts by selecting paths, credentials, release and input files. Later review happens through Error Cases and DB inspection.

Failure branches

- Missing or invalid active Semantic Release.
- Input file unsupported by the Optimizer.
- Model request failure.
- Validator rejects the stage output.
- Normalizer cannot map content into the active projection.
- Corpus Builder rejects materialization because release, schema or target proof does not match.

Recovery strategy

Use Error Cases for rejected evidence, fix configuration or release state, then rerun or reimport intentionally. Do not manually patch stage JSON and pretend the pipeline produced it.

Expected end state

The Corpus DB contains materialized documents for successful inputs, Error Cases contain failed evidence, and the Orchestrator run summary matches the stage artifacts.

7. Manual Pipeline Run Workflow

Purpose

Run the same ingestion path through the Taxonomy Agent and Kernel. This is the agent-facing production ingest workflow.

Trigger

The Taxonomy Agent calls `manual_pipeline_run`.

Preconditions

- Target artifact tree exists.
- Target Corpus DB exists and is bound to the artifact tree.
- Target DB has an active Semantic Release.
- Input evidence exists.
- The user confirms the target and input set through Kernel interaction dialogs.

Steps

1. Kernel collects or resolves artifact root and DB name.
2. Kernel proves the DB/artifact binding and active release state.
3. Kernel creates or reuses a `pipeline_batch_id`.
4. Kernel writes a pending batch manifest.
5. Kernel calls the Pipeline Batch adapter.
6. The Orchestrator owner creates the owner-side batch manifest and starts the pipeline.
7. Kernel polls snapshot progress through the progress bridge.
8. Orchestrator executes the stage order from the execution policy.
9. Kernel correlates owner output with the pending batch manifest.
10. Kernel finalizes the batch manifest and writes the final notice.

Produced artifacts

- Pending and final Kernel batch manifests.
- Owner pipeline batch manifest.
- Correlation report.
- Orchestrator stage artifacts.
- Corpus DB materialization.
- Final notice with DB path, artifact root, input count, materialized document count, review count and error case summary.

Human review points

The user confirms the artifact root, DB and input set. If the workflow offers Error Case restore or recovery choices, those choices must be answered through Kernel dialogs.

Failure branches

- No active Semantic Release.
- DB path does not stay inside the artifact tree storage boundary.
- Input evidence is missing.
- Owner pipeline run fails.
- Correlation fails because expected and actual owner output disagree.
- Final notice cannot be emitted because the final manifest is incomplete.

Recovery strategy

Use `kernel_status` to see whether the run is active, blocked or resumable. If the Orchestrator produced Error Cases, inspect those before retrying. If the pipeline owner failed before materialization, fix the owner blocker and rerun.

Expected end state

The user receives a final Taxonomy Agent notice. The final notice is the completion signal, not the last visible stage counter.

8. Ingestion Stage Micro-Workflows

The ingestion pipeline is best understood as a chain of micro-workflows. Each stage owns a narrow responsibility and writes evidence for the next stage.

Stage	Purpose	Durable Handoff
Intake	Discover source files and schedule records/pages.	Pipeline state and page lifecycle entries.
Runtime Semantics	Load active taxonomy, projections and release metadata.	Runtime semantic assets used by downstream stages.
Optimizer	Convert files into raw locatable evidence without domain semantics.	Raw JSON, page images and extraction references.
Request Enrichment	Build model request payloads for interpretation.	Persisted Interpreter request JSON.
Interpreter	Describe what is visible in the evidence.	Structured interpretation output and classification hints.
Projection Hint Validation	Score Interpreter hints against local structured signals.	Hint validation result used before normalization.
Validator	Check that stage output is safe enough to continue.	Validated or rejected stage state.
Normalizer	Map interpretation into active taxonomy/projection structures.	Normalized JSON and persisted Normalizer requests.
Corpus Builder	Materialize normalized content into SQLite.	Documents, pages, chunks, evidence atoms, page images and release-bound rows.
Embeddings	Create vector search material when configured.	Embedding rows and embedding status.

Single-file and multi-page documents follow the same stage chain. Multi-page state is managed by the scheduler and later made easier to query by the Base Graph. Page-level processing remains important because page images are the immutable visual evidence from which materialized DB content can be checked.

9. Database Creation Workflows

The six empty database creation workflows all share the same base structure:

1. Collect target artifact tree and DB name.
2. Create the artifact tree if needed.
3. Store the artifact-tree binding.
4. Create the empty Corpus DB through the Corpus Builder owner.
5. Attach or author Semantic Release components depending on the selected route.
6. Activate a complete release if the route produces one.
7. Emit a final notice and, when incomplete by design, resumable next steps.

Tool	Taxonomy	Projections	Final State	Typical Use
<code>empty_database_no_semantic_release</code>	None	None	<code>no_semantic_release</code>	Low-level scaffold or support case.
<code>empty_database_default_taxonomy_no_projections</code>	Default	None	<code>semantic_release_incomplete</code>	Prepare a DB before custom projection authoring.
<code>empty_database_default_taxonomy_default_projections</code>	Default	Default	<code>semantic_release_active</code>	Fastest ready-to-ingest database.
<code>empty_database_default_taxonomy_custom_projections</code>	Default	Custom	<code>semantic_release_active</code>	Keep the default taxonomy but adapt extraction views.
<code>empty_database_custom_taxonomy_no_projections</code>	Custom	None	<code>semantic_release_incomplete</code>	Author a domain taxonomy first, projection later.
<code>empty_database_custom_taxonomy_custom_projections</code>	Custom	Custom	<code>semantic_release_active</code>	Build a domain-specific ready-to-ingest database.

Preconditions

- The user can provide a target location and DB name.

- The target path does not collide with an existing DB unless the route has an explicit safe continuation.
- Custom taxonomy or projection paths require sample evidence and LLM credentials.

Produced artifacts

- Artifact tree folders.
- Empty Corpus DB.
- Semantic Release package files when a release exists.
- Kernel route state and final notice.
- Resumable options when the route intentionally stops incomplete.

Failure branches

- Missing target input.
- Existing DB or sidecar files where the route expects a clean target.
- Target identity drift.
- Missing or invalid sample evidence for custom authoring.
- LLM authoring failure.
- Semantic Release activation preflight failure.

Recovery strategy

If the workflow stopped after creating a DB but before release completion, use `kernel_resume_state`. Custom taxonomy and projection continuation routes must be resumed from a valid Kernel resume option; they are not free-standing "invent a path from chat" workflows.

Code anchors

- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/database_creation/route_catalog.py
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/database_creation/route_steps.py
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/database_creation/route_resume.py

- 05 - Corpus Builder/corpus_builder/services/corpus_context.py

10. Custom Taxonomy Authoring Workflow

Purpose

Create a taxonomy component from user-selected samples and user intent.

Trigger

The user selects a custom-taxonomy database creation route, or resumes a creation workflow that stopped before projection authoring.

Preconditions

- Sample files are available.
- LLM credentials are configured.
- Target artifact tree and DB state are already known to the Kernel route.

Steps

1. Kernel asks for sample selection.
2. Orchestrator/Kernel sample handling proves that samples still match the expected target.
3. The authoring LLM analyzes sample evidence.
4. The workflow creates a taxonomy proposal.
5. The route builds update state and materializes the custom taxonomy component.
6. The component is staged for a Semantic Release.

Produced artifacts

- Taxonomy proposal/update state.
- Custom taxonomy component.
- Staged release state or incomplete resume state.

Human review points

The user decides whether the sample set is representative enough and whether to continue toward projections immediately or later.

Failure branches

- No samples selected.
- Sample identity changed.
- LLM output fails final validation.
- Taxonomy component cannot be materialized.

Expected end state

The route either stages a valid taxonomy for projection authoring or stops with a resumable incomplete state.

11. Custom Projection Authoring Workflow

Purpose

Create projection components that bind the taxonomy to a concrete extraction and normalization view.

Trigger

The user selects a route with custom projections or resumes a DB creation route that needs projections.

Preconditions

- A taxonomy proof exists, either default or custom.
- Projection samples are available.
- LLM credentials are configured.
- Target DB and artifact tree are still bound to the route state.

Steps

1. Kernel resolves the taxonomy proof.
2. Kernel asks for sample evidence.
3. The workflow builds a projection authoring view.
4. The authoring LLM proposes projection structure.
5. The workflow materializes projection components.
6. Projection bindings are validated against the taxonomy.
7. A complete custom release can be created, written, attached and activated.

Produced artifacts

- Projection proposal/update state.
- Projection component files.
- Custom Semantic Release package when taxonomy and projections are complete.

Failure branches

- Missing taxonomy proof.
- Invalid projection binding.
- Final validation failure.
- Activation preflight rejects the release.

Expected end state

Either a complete active Semantic Release exists, or the DB remains in an explicit incomplete state with resumable context.

12. Semantic Release Publication And Activation Workflow

Purpose

Turn taxonomy and projection components into a release package and make that release active for a Corpus DB.

Trigger

Release activation can happen during default DB creation, custom DB creation, merge, rebuild or direct Orchestrator release activation.

Preconditions

- Taxonomy and projections have stable fingerprints.
- The release identity is complete.
- Activation is safe for the target DB state.
- Any required user confirmation has been collected.

Steps

1. Build or export release components.
2. Write release files to the artifact tree.
3. Attach release state to the Kernel/target context.
4. Run Corpus Builder activation preflight.
5. Apply the release snapshot inside the Corpus DB.
6. Roll back if activation throws an exception.

Produced artifacts

- Semantic Release package.
- Active release snapshot in the Corpus DB.
- Activation receipt or blocker.

Failure branches

- Missing taxonomy or projection fingerprint.
- Incompatible active documents.
- Missing confirmation for unsafe activation.
- Corpus Builder apply failure.

Expected end state

The Corpus DB has exactly one active release snapshot. Ingestion can now start.

13. Embedding Generation Workflow

Purpose

Create vector search material for the Corpus DB after ingestion or rebuild.

Trigger

Embedding generation is invoked by Orchestrator downstream actions or by the Kernel rebuild workflow. It is not one of the sixteen permanent Taxonomy Agent workflow-start tools.

Preconditions

- The Corpus DB exists.
- There is materialized content that needs embeddings.
- A supported embedding provider is configured, unless the calling workflow is allowed to skip missing embeddings.

Steps

1. Resolve pending source text/chunks.
2. Call the embedding provider.
3. Store embedding vectors and embedding metadata.
4. Report counts or provider status.

Failure branches

- Provider credentials are missing.
- Provider request fails.
- Vector storage fails.
- Strict embedding policy is enabled and no provider exists.

Recovery strategy

Under default policy, missing embedding credentials can be a warning. Under strict policy it is a blocker. Once credentials are fixed, rerun the embedding action or rebuild path that invokes it.

Expected end state

The DB has embedding rows for eligible materialized chunks, or a documented embedding-unavailable state.

14. Database Rebuild From Artifacts Workflow

Purpose

Rebuild a Corpus DB from artifact-level evidence instead of re-running the full input pipeline.

Trigger

The Taxonomy Agent calls `database_rebuild_from_artifacts`.

Preconditions

- Artifact root exists.
- Semantic Release artifacts exist.
- Target DB name is valid.
- Existing DB overwrite is explicitly confirmed if required.

Steps

1. Kernel collects artifact root and target DB name.
2. Kernel normalizes target path under the artifact tree's `Corpus` area.
3. Kernel loads the Semantic Release from artifacts.
4. Kernel asks for overwrite confirmation if the DB already exists.
5. Corpus Builder scans standalone artifacts.
6. Corpus Builder validates payloads against the release.
7. Corpus Builder replaces or creates DB files.
8. The rebuild path seeds the release snapshot.
9. Embeddings run or are skipped according to policy.
10. The release is activated and a rebuild manifest is written.

Produced artifacts

- Rebuilt Corpus DB.
- Rebuild manifest.
- Activation receipt.
- Embedding status.
- Final notice.

Failure branches

- Missing Semantic Release package.
- Invalid target path or DB name.

- Existing DB without matching overwrite receipt.
- Rebuild release fingerprint mismatch.
- Strict embedding policy failure.
- Activation failure.

Expected end state

The target DB can be used as a normal active Corpus DB and points back to the artifact tree evidence from which it was rebuilt.

15. Database Reset Workflow

Purpose

Clear materialized Corpus content while preserving the active Semantic Release.

Trigger

The Taxonomy Agent calls `reset_database`.

Preconditions

- Target artifact tree and DB are known.
- DB exists and has an active release snapshot.
- The user gives destructive confirmation through a Kernel dialog.

Steps

1. Kernel collects artifact root and DB name.
2. Kernel asks for destructive confirmation.
3. Kernel proves target identity.
4. Corpus Builder clears materialized content tables and search indexes.
5. Corpus Builder proves the Semantic Release is preserved.
6. Kernel writes a reset manifest and final notice.

Produced artifacts

- Reset manifest under the artifact tree logs area.
- Empty materialized DB content.
- Preserved Semantic Release snapshot.

Failure branches

- Missing DB.
- Missing, negative or stale confirmation.
- Target identity drift.
- No active release to preserve.
- Corpus Builder reset proof fails.

Expected end state

The DB is ready for a fresh ingest using the preserved active release. Source artifacts and Semantic Release files are not deleted.

16. Additive Database Merge Workflow

Purpose

Merge multiple source databases into a new target without mutating or deleting the source databases.

Trigger

The Taxonomy Agent calls `database_merge_additive_only`.

Preconditions

- At least two source databases are selected.
- Source fingerprints and release states are readable.
- Target artifact root and DB are not one of the sources.
- The selected projection merge mode is compatible with the sources.
- Non-empty or conflicting targets require explicit confirmation.

Steps

1. Kernel asks for source count and source paths.
2. Kernel validates source fingerprints and target safety.
3. Kernel asks for target artifact tree and DB name.
4. Kernel classifies the merge route.
5. Empty-source merge routes merge Semantic Release state.
6. Filled-source merge routes copy SQL rows and artifacts additively.
7. ID maps are written so copied rows preserve relationships.
8. Materialization references are validated.
9. Taxonomy, projections, Base Graph and ontology lenses are merged.
10. Collision manifests are built when semantic conflicts exist.
11. User reconciliation is requested if needed.
12. The merged release is attached and activated.
13. Locks are released and a final notice is emitted.

Produced artifacts

- Target artifact tree.
- Target Corpus DB.
- Merge selection and route state.
- Merge receipts and ID maps.

- Collision manifest when needed.
- Final notice.

Failure branches

- Invalid source selection.
- Target equals source.
- Target root is unsafe or conflicting.
- Mixed source states cannot be routed.
- SQL copy fails.
- Too many unresolved semantic collisions.
- Ontology parent cycles or invalid merged ontology state.
- Owner response exceeds Kernel response limits.

Recovery strategy

For user-reconcilable collisions, answer the Kernel reconciliation dialog. For support-only blockers, inspect merge receipts and source DB integrity before starting a new merge. Do not hand-edit the target into a pretend-complete state.

Expected end state

The target DB contains additive content from all sources, has an active release when the merge route can produce one, and preserves merge provenance.

17. Base Graph Mining Workflow

Purpose

Create deterministic structural relations after page-level materialization. This is the first layer that makes page-wise DB content behave like source documents again.

Trigger

The Ontology Agent calls `basic_relation_mining`, or Kernel code invokes the same Corpus Builder ontology primitive.

Preconditions

- The active configured DB exists.
- Corpus Builder ontology schema can be ensured.
- The DB contains materialized documents/pages.

Steps

1. Resolve the active DB path from the configured agent context.
2. Ensure ontology/Base Graph schema.
3. Group pages by explicit source-document identifiers.
4. Populate source-document and source-document-page structures.
5. Populate deterministic structural relations.
6. Report counts, warnings and unresolved ambiguities.

Produced artifacts

- Base Graph/source-document rows in the Corpus DB.
- Structural relations.
- Mining report.

Human review points

There should be no LLM judgement in this workflow. Ambiguity is reported rather than guessed.

Failure branches

- DB path missing.
- Schema cannot be ensured.
- Required source identifiers are absent.
- Output DB proof does not match the requested target.

Expected end state

The Query Agent and Ontology Agent can see source-document grouping without having to infer it from page rows at answer time.

18. Ontology Agent Workflow

Purpose

Let the user create, compare and revise evidence-bound knowledge lenses over an already materialized Corpus DB.

Trigger

The user selects the Ontology Agent and asks for a lens, correction, comparison or mining task.

Preconditions

- A Corpus DB is configured in the Client Frontend.
- The DB schema includes the ontology layer.
- The user intent is clear enough to start, or the agent asks clarifying ontology-design questions.

Available tool pattern

The Ontology Agent reuses Query Agent read tools and adds controlled write tools. Its document reads should normally use the compact `get_document_*` views before escalating to the full document bundle.

- `basic_relation_mining`
- `sql_batch_execute`

`sql_batch_execute` is the write gate. It is expected to preflight statements, repair missing required identifiers before writing, run validation after writes, retry bounded repair attempts and refresh ontology embeddings when possible. The tool does not accept a free DB path; it writes to the configured active DB.

Steps

1. Inspect the DB and existing lenses.
2. Help the user turn fuzzy intent into an explicit lens design.
3. Read source documents, ontology state and evidence.
4. Create or update lens metadata.
5. Create terms, nodes, edges, assertions and evidence links with stable IDs.
6. Validate DB constraints and ontology object identifiers.
7. Refresh ontology embeddings if credentials exist.
8. Explain what changed and which evidence supports it.

Produced artifacts

- Ontology lenses.
- Ontology terms, nodes, edges and assertions.
- Evidence links to materialized DB evidence.
- Ontology embedding chunks when available.

Failure branches

- Missing required object IDs.
- Foreign-key order violation.
- Missing non-null JSON attributes.
- Invalid lens status.
- Evidence link points to a non-existing object.
- Embedding provider unavailable.

Recovery strategy

The agent should repair predictable write-shape failures inside the tool loop before bothering the user. User intervention is for semantic decisions, not for basic SQL column discipline.

Expected end state

The DB contains a usable lens that can be read by the Query Agent without overwriting base facts.

19. Query Agent Workflow

Purpose

Answer questions over the configured Corpus DB while exposing source evidence and taking ontology lenses into account.

Trigger

The user asks a question in the Query Agent tab.

Preconditions

- Frontend configuration points to a readable Corpus DB.
- The DB has materialized content.
- If ontology lenses exist, the Query Agent should treat them as part of the DB meaning, not as optional decoration.

Available read tools

- `sql_query`
- `get_document_summary`
- `get_document_ontology_evidence`
- `get_document_rows`
- `get_document_provenance`
- `get_document_full`
- `get_document`
- `get_provenance`
- `semantic_search`
- `database_coverage_snapshot`
- `list_source_documents`
- `get_source_document`
- `list_ontology_lenses`
- `get_ontology_lens`
- `workbench`

The `get_document_*` family is the preferred document escalation path. The legacy `get_document` full read remains available for compatibility and last-resort inspection.

Steps

1. Inspect coverage when needed.
2. Use SQL, document reads, provenance and semantic search as appropriate.

3. Inspect active ontology lenses when they can affect the answer.
4. If an active correction, audit or review lens contradicts materialized facts, surface the contradiction instead of hiding it.
5. Produce an answer with source references.
6. The Frontend reconciles answer-mentioned sources against documents read during the turn.

Produced artifacts

- Answer text.
- Turn source list.
- Source reconciliation state in the UI.

Failure branches

- DB unavailable.
- Source mentioned in the answer cannot be resolved against the turn source list.
- Semantic search unavailable because embeddings are missing.
- The user asks for a claim outside the DB evidence.

Expected end state

The user receives an evidence-grounded answer. If a source link is suspicious, the UI should make that visible instead of silently trusting the model.

20. Error Case Inspection Workflow

Purpose

Preserve failed evidence in a form that can be inspected, debugged and rerun without losing the original context.

Trigger

A pipeline stage rejects an input, page or record; or the user opens Error Case inspection through Orchestrator/MCP surfaces.

Preconditions

- The pipeline has written Error Case bundles or review flags.
- The artifact tree logs and error folders are accessible.

Steps

1. Locate the Error Case bundle.
2. Inspect the source evidence, stage request, model response and validator diagnostic.
3. Determine whether the failure is input-related, model-related, taxonomy/projection-related or materialization-related.
4. Fix the correct owner surface.
5. Retry or reimport through a manifest-backed path.

Produced artifacts

- Error Case bundle.
- Retry/reimport manifest when the evidence is reintroduced.
- Updated pipeline or DB state after successful rerun.

Failure branches

- Error bundle is incomplete.
- Retry evidence no longer matches target identity.
- The failure is caused by unsupported input type or architectural limitation.

Expected end state

The failed evidence is either successfully reprocessed or explicitly preserved as a known unsupported/error case.

21. Recovery And Resume Workflow

Purpose

Continue, cancel or diagnose workflows that stopped between user intent and final notice.

Trigger

The user asks what happened, a workflow blocks, a Kernel dialog is pending, or the UI shows background work without a final notice.

Preconditions

- Kernel state root is readable.
- The Frontend can call Kernel support tools.

Steps

1. Call `kernel_status`.
2. If a dialog is pending, answer or cancel the dialog through the Kernel UI.
3. If no dialog is pending but resume options exist, call `kernel_resume_state`.
4. Pick the appropriate opaque resume option.
5. Call `kernel_continue_resumable_workflow` with only the `resume_option_ref`.
6. If the active run should not continue, call `kernel_cancel_active_run`.

Produced artifacts

- Updated Kernel state.
- Resumed workflow events or cancelled active state.
- Final notice or blocker explanation.

Failure branches

- Resume option is stale.
- The underlying owner artifact was moved or edited.
- The workflow is support-only unrecoverable.
- The user cancels an interaction required for a destructive or target-changing action.

Expected end state

The workflow is completed, resumed into a new pending user decision, explicitly blocked with a reason, or cancelled.

22. Testing And Verification Map

Workflow verification should follow the same boundary structure as the runtime:

- **Tool inventory tests** prove the permanent Taxonomy Agent tool list.
- **Dispatcher tests** prove public tools route to the intended workflow or support handler.
- **Route tests** prove final states, blockers and resumable options.
- **Owner adapter tests** prove that Kernel requests remain inside the owner contract.
- **Artifact tree tests** prove paths, manifests and DB placement.
- **Corpus Builder tests** prove DB schema, release activation, rebuild, reset, merge and Base Graph mining.
- **Frontend tests** prove workflow menu behavior, Kernel event polling, interaction dialogs and source reconciliation.
- **End-to-end smoke runs** prove direct Orchestrator ingest, Kernel manual ingest, rebuild, merge and ontology/query reading against real sample DBs.

The useful rule for future maintenance is simple: test the boundary that owns the risk. Do not patch over a workflow failure in the Frontend when the owner proof failed in Corpus Builder, and do not relax a Kernel guard because a UI label made an internal continuation path look like a normal button.

The whole chapter in one line

Test the boundary that owns the risk: Kernel for route state, owner modules for mutation, artifacts and database rows for proof.